

CS342: Computer Networks Lab
(July–November 2025)
Assignment - 03

Submission Deadline : 17th October, 11:55 pm,

In this assignment you need to simulate a computer network for a given application using the discrete event network simulator ns-3. You can download the software and documentation of ns-3 from the website <https://www.nsnam.org>. The assignment will be solved in groups where each group, comprised of 4 members, needs to work on an application assigned to it. The applications' network specifications, the required experiments, and related questions are given in pages 2–6 of this document. Follow the general instructions given below and any specific instructions mentioned in the application description.

General Instructions

1. For each group, Problem 1 and Problem 4 are mandatory. All even numbered groups are additionally assigned Problem 2. All odd numbered groups are additionally assigned Problem 3. Only one member from a group needs to make the submission.
2. Install ns-3 on your computer, write programs and simulate the network described in the given application assigned to you, perform the required experiments, and answer the given questions.
3. Use ns-3's Flow Monitor module to collect and store the performance data of network protocols from the simulation. **Do not use PCAP + Wireshark for the trace collection. No marks will be awarded if your application uses PCAP file for trace collection.**
4. Submit your set of source code files, along with a report clearly mentioning all observations, explanations, containing the graphs, and all the answers, as a zipped file on the form. The ZIP file's name should be the same as your group number, for example, "Group_3.zip", or "Group_3.rar", or "Group_3.tar.gz". **Submissions without a report will not be awarded any marks.**
5. Write your own source codes and do not copy from any source. Plagiarism and use of unfair means will be penalised by awarding **NEGATIVE marks (equal to the maximum marks for the assignment)**.

Problem 1: Reliable Data Transfer using NS-3

Objectives

- Understand the principles of reliable data transfer (RDT) protocols.
- Implement Alternating Bit Protocol (Stop-and-Wait) and Go-Back-N using NS-3.
- Learn to simulate packet loss and corruption in a realistic network simulator.

- Collect performance metrics (throughput, retransmissions, delay) and analyze results.

Background

Reliable data transfer protocols ensure correct, in-order delivery of packets despite an unreliable network. Traditional implementations include:

- **Alternating Bit Protocol (ABP / Stop-and-Wait)** – sender transmits one packet at a time, waits for acknowledgment (ACK/NAK).
- **Go-Back-N (GBN)** – sender can transmit multiple packets within a sliding window, but must retransmit all packets from the point of loss.

In this lab, instead of using a custom emulator, you will implement and test these protocols in **NS-3**, a discrete-event network simulator widely used for research and teaching.

Assignment Tasks

Part A: Alternating Bit Protocol

1. Implement a Stop-and-Wait protocol in NS-3 using two nodes connected via a point-to-point link.
2. The sender transmits data packets (simulate with `OnOffApplication` or a custom application).
3. The receiver sends ACK/NAK packets using a custom header.
4. Introduce artificial packet loss and corruption using the `RateErrorModel`.
5. Collect and plot:
 - Number of retransmissions vs. simulation time.
 - Throughput vs. simulation time.

Part B: Go-Back-N Protocol

1. Extend your implementation to allow a sender window size of N (e.g., $N = 4$).
2. Implement cumulative acknowledgments at the receiver.
3. Use a single timer for retransmissions.
4. Collect and plot:
 - Window utilization vs. time.
 - Packet loss rate vs. retransmissions.
 - Goodput (useful bytes delivered) vs. simulation time.

Simulation Setup in NS-3

- **Topology:** Two nodes connected by a point-to-point channel.
- **Link parameters:** Bandwidth = 1 Mbps, Delay = 10 ms.
- **Error model:** Use `RateErrorModel` with configurable packet loss and corruption probabilities.
- **Simulation time:** 20 seconds.

Deliverables

- NS-3 source code implementing ABP (`abp.cc`) and GBN (`gbn.cc`).
- A PDF report containing:
 1. FSM diagrams for sender and receiver (for both ABP and GBN).
 2. Explanation of packet formats (headers used for seqnum, acknum, checksum).
 3. Graphs of results (retransmissions, throughput, goodput).
 4. Discussion comparing ABP vs. GBN performance under loss/corruption.

Evaluation

- Correct implementation of ABP: **40%**.
- Correct implementation of GBN: **40%**.
- Report quality (design explanation, graphs, discussion): **20%**.

Problem 2: TCP Congestion Control

Problem Statement:

Create a topology of two nodes N0 and N1 connected by a link of bandwidth 1 Mbps and link delay 10 ms. Use a drop-tail queue at the link. Set the queue size according to the bandwidth-delay product. Create a TCP agent (type of the agent specified below) and FTP traffic at N0 destined for N1. Create 5 CBR traffic agents of rate 500 Kbps each at N0 destined for N1. Make appropriate assumptions wherever necessary. The timing of the flows are as follows:

- FTP starts at 0 sec and continues till the end of simulation.
- CBR1 starts at 300 ms and continues till end.
- CBR2 starts at 500 ms and continues till end.
- CBR3 starts at 700 ms and stops at 1100 ms.
- CBR4 starts at 800 ms and stops at 1500 ms.
- CBR5 starts at 1000 ms and stops at 1600 ms.

- Simulation runs for 2000 ms.

Tasks:

1. Plot graph(s) of TCP congestion window with respect to time for the following 6 TCP congestion control algorithm implementations, and describe their behaviour:
 - (a) TCP VenO
 - (b) TCP Hybla
 - (c) TCP New Reno
 - (d) TCP HighSpeed
 - (e) TCP Vegas
 - (f) TCP Westwood
2. Draw a graph showing cumulative TCP packets dropped with respect to time, comparing the above 6 TCP congestion control algorithms.
3. Draw a graph showing cumulative bytes transferred with respect to time, comparing the above 6 TCP congestion control algorithms.

Problem 3: HTTP Web Proxy Server

Implement and analyze the performance of an **HTTP Web Proxy Server**. The proxy server should act as an intermediary between clients and web servers, forwarding HTTP requests and caching responses to improve performance.

Network Setup

- One node acts as the **proxy server**.
- Multiple client nodes send HTTP GET requests through the proxy.
- Web servers provide the requested content.

Tasks

1. **Basic Proxy Functionality:**
 - Implement a proxy server that listens on a given port, accepts client requests, forwards them
 - **Caching Mechanism:**
 - Cache web objects (responses) locally at the proxy.
 - If the same object is requested again, serve it from the cache instead of contacting the server.
 - Record cache hits and misses.
 - **Performance Analysis:**
 - Measure average response time with and without caching.

- Measure cache hit ratio for repeated requests.
- Evaluate throughput and latency when multiple clients connect simultaneously.

Experiments

- Use at least 5 distinct client requests to different websites/objects.
- Perform experiments with repeated requests to demonstrate caching efficiency.
- Vary the number of clients (e.g., 1, 5, 10) and record the performance metrics.

Expected Results

- Graph of **response time vs. number of clients**.
- Graph of **cache hit ratio vs. number of requests**.
- Graph of **throughput vs. simulation time**.
- Discussion of how caching improves performance.

Problem 4 : Distance Vector Routing Algorithm and Count-to-Infinity problem

In this, you will implement using C++ and analyze the **Distance Vector Routing Algorithm** and observe the well-known **Count-to-Infinity problem**.

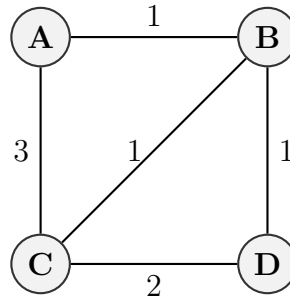
Conceptual Background. Distance vector routing is a distributed algorithm where each router maintains a table containing the best-known distances (costs) to all other routers in the network and the corresponding next hops. Each router periodically exchanges its distance vector with immediate neighbors, updating its own table using the **Bellman-Ford equation**:

$$D_x(y) = \min_{v \in N(x)} \{c(x, v) + D_v(y)\}$$

where $D_x(y)$ is the cost from node x to node y , $N(x)$ is the set of neighbors of x , and $c(x, v)$ is the link cost between x and neighbor v .

This iterative process continues until all nodes have consistent routing tables. However, when a link fails, incorrect information may propagate slowly through the network, causing the well-known **Count-to-Infinity problem**, where routers continuously increase their distance to an unreachable node without quickly realizing it is no longer accessible.

Network Topology. Consider the following network of four routers:



Initially, all routers exchange distance vectors until routing tables converge. Then, simulate a failure of the link between nodes B and D and observe how routing tables evolve over time.

Simulate the iterative behavior of the Distance Vector Routing algorithm in code. The simulation does not need to use real network packets; instead, focus on updating routing tables and exchanging distance vectors in discrete rounds.

- Represent the network as an adjacency matrix or list of link costs.
- Initialize each router's distance vector with direct link costs to its neighbors; all other distances start as infinity.
- Implement iterative updates based on the Bellman-Ford equation until routing tables converge.
- After convergence, simulate a **link failure** (e.g., between B and D) by removing that link (set its cost to infinity).
- Continue running update rounds after the failure to observe how incorrect routes propagate and distances increase — illustrating the Count-to-Infinity phenomenon.

Implement and demonstrate one or more of the following techniques to mitigate the Count-to-Infinity problem:

- **Split Horizon:** Do not advertise a route back on the interface from which it was learned.
- **Poison Reverse:** Advertise a failed route back to the sender with an infinite metric.

Compare how these mechanisms reduce the convergence delay or prevent counting to infinity.

Pseudocode Guidance. A simplified structure of the simulation is as follows:

```
# Initialization
for each node x:
    for each destination y:
        if y is a neighbor of x:
```

```

        D[x][y] = cost(x, y)
    else:
        D[x][y] = infinity

# Iterative updates
repeat until convergence:
    for each node x:
        for each neighbor v:
            for each destination y:
                if D[x][y] > cost(x,v) + D[v][y]:
                    D[x][y] = cost(x,v) + D[v][y]

```

After convergence, simulate link failure (e.g., set $\text{cost}(B,D) = \text{infinity}$) and continue iterations. Record how distances change across rounds.

Analysis and Discussion. Include the following in your analysis:

- The initial routing tables at convergence (before failure).
- The evolution of routing tables over subsequent rounds after link failure.
- The number of rounds required for the network to reach stability again.
- An explanation of why Count-to-Infinity occurs and how mitigation techniques improve convergence.